PROJECT ROADMAP

Virtual Customer Care Office

(VCC)

Self-Evolving AI-Powered Feedback System

Testing & Proof of Concept Phase
Estimated Timeline: 2 Weeks

March 2, 2026

Table of Contents

Table of Contents 2
Executive Summary 4
Project Overview 4
Project Objectives 4
Key Stakeholders 4
Framework Evaluation 4
Framework 1: NanoBot 5
Overview 5
Key Features 5
Best For 5
Framework 2: Opencode SDK 5
Overview 5
Key Features 5
Best For 5
Framework 3: CrewAI 6
Overview 6
Key Features 6
Best For 6
Framework 4: LangGraph 6
Overview 6
Key Features 6
Best For 6
Framework Comparison Matrix 6
Phase 1: Competitive Framework Testing 7
Objective 7
Duration 7
Module 1: Check-In Module 7
Functional Requirements 7
Module 2: Check-Out Module 7
Functional Requirements 7
Phase 2: Self-Evolving Feedback System 8
Objective 8
Duration 8
24-Hour Review Cycle 8
Dynamic Form Adaptation 8
Add Questions 8

Remove Questions 8
Contextual Intelligence 8
Phase 3: Internal Analytics Dashboard 8
Objective 8
Duration 9
Dashboard Components 9
Logic Visualization 9
Sentiment & Trend Analysis 9
Operational Insights 9
Phase 4: Evaluation & Selection 9
Objective 9
Duration 9
Evaluation Criteria 9
Performance Metrics 9
Scalability Assessment 10
Technical Architecture 10
Technology Stack 10
Project Timeline 10
Risk Assessment 11
Success Metrics 11
Quantitative Metrics 11
Qualitative Metrics 11
Conclusion 11

Note: Right-click the Table of Contents and select "Update Field" to refresh page numbers.

Executive Summary

The Virtual Customer Care Office (VCC) represents a groundbreaking approach to customer service management, combining artificial intelligence with dynamic feedback systems to create an autonomously improving customer experience platform. This project aims to develop a proof-of-concept system that demonstrates the feasibility of AI-driven form adaptation and real-time sentiment analysis within a two-week sprint timeline.

The core innovation lies in the Self-Evolving Feedback System, where AI continuously analyzes customer interactions and autonomously modifies data collection instruments based on emerging patterns, recurring issues, and identified gaps in information. This creates a "living" feedback mechanism that becomes increasingly intelligent and contextually aware over time, ultimately delivering more actionable insights for business improvement.

This roadmap outlines a structured four-phase approach designed for competitive framework testing, allowing team members to evaluate multiple AI frameworks (NanoBot, Opencode SDK, CrewAI, and LangGraph) through practical implementation. The ultimate goal is to identify the most effective framework for scalability, performance, and human-like AI interactions.

Project Overview

Project Objectives

1. Build a functional Virtual Agent that manages the end-to-end client "Check-in" and "Check-out" experience through intuitive web-based interfaces.
2. Implement AI-driven sentiment analysis and pattern recognition to automatically identify trends and issues in customer feedback.
3. Create a dynamic form adaptation system that autonomously adds, modifies, or removes questions based on AI analysis of feedback patterns.
4. Develop a comprehensive analytics dashboard providing visibility into AI decision-making logic and operational insights.
5. Compare multiple AI frameworks to determine optimal performance, scalability, and development efficiency for future production deployment.

Key Stakeholders

Stakeholder	Role	Responsibility
Development Team	Implementation	Build prototypes using assigned frameworks
Project Manager	Coordination	Timeline management and resource allocation
Business Analysts	Requirements	Define success metrics and evaluation criteria
End Users	Testing	Provide feedback on usability and experience

Table 1: Key Project Stakeholders

Framework Evaluation

Each team member will select one framework from the options below to build a functional prototype of the VCC. This parallel development approach enables direct comparison of framework capabilities, development velocity, and AI reasoning quality under identical requirements. The evaluation focuses on four key frameworks, each with distinct architectural philosophies and strengths.

Framework 1: NanoBot

Overview

NanoBot is a lightweight, minimalist AI agent framework designed for rapid prototyping and micro-service architectures. Its core philosophy emphasizes simplicity and speed, making it ideal for proof-of-concept projects where quick iteration is essential. The framework provides essential building blocks without imposing heavy abstractions, allowing developers to maintain fine-grained control over agent behavior.

Key Features

- Minimal Overhead: Extremely lightweight core with minimal dependencies, resulting in fast startup times and low memory footprint
- Flexible Architecture: Unopinionated design allows custom implementations without framework constraints
- Rapid Prototyping: Built-in utilities for quick agent creation and testing workflows
- Easy Integration: Simple API surface that integrates well with existing codebases

Best For

Teams prioritizing development speed and simplicity, projects requiring minimal infrastructure overhead, and scenarios where custom agent logic is paramount. Ideal for developers who prefer explicit control over implicit automation.

Framework 2: Opencode SDK

Overview

Opencode SDK provides a comprehensive toolkit for building production-ready AI agents with enterprise-grade features. It emphasizes code generation capabilities and structured outputs, making it particularly suited for applications requiring reliable, predictable AI behavior. The framework bridges the gap between prototyping and production deployment.

Key Features

- Code Generation: Native support for generating, validating, and executing code within agent workflows
- Structured Outputs: Strong typing and schema validation for AI responses ensuring data integrity
- Enterprise Ready: Built-in security features, rate limiting, and observability tools
- Comprehensive SDK: Rich API covering common use cases with sensible defaults

Best For

Projects requiring structured, validated outputs and enterprise-grade reliability. Excellent for teams planning rapid transition from prototype to production, and applications where AI-generated code execution is a core requirement.

Framework 3: CrewAI

Overview

CrewAI specializes in multi-agent orchestration, enabling teams of AI agents to collaborate on complex tasks. Its unique value proposition lies in defining agent roles, goals, and inter-agent communication patterns. This framework excels in scenarios where different AI capabilities need to work together, mimicking human team structures.

Key Features

- Multi-Agent Orchestration: Define and coordinate multiple specialized agents working together
- Role-Based Design: Agents have defined roles, goals, and backstories for contextual behavior
- Task Delegation: Sophisticated task routing and delegation between agents
- Collaborative Memory: Shared context and memory between team members

Best For

Complex workflows requiring specialized AI capabilities to work in concert. Ideal for projects where different aspects of customer service (sentiment analysis, form generation, insights synthesis) benefit from dedicated, specialized agents with clear responsibilities.

Framework 4: LangGraph

Overview

LangGraph, built on LangChain, provides a graph-based approach to building stateful, multi-actor applications with LLMs. It excels in defining complex workflows with explicit state management, conditional branching, and cycles. This framework offers maximum control over agent execution flow and decision-making processes.

Key Features

- Graph-Based Workflows: Define agent logic as directed graphs with nodes and edges
- State Management: Built-in state persistence and checkpointing for complex conversations
- Conditional Branching: Dynamic routing based on intermediate results and conditions
- LangChain Integration: Seamless access to LangChain ecosystem of tools and integrations

Best For

Applications requiring complex, stateful workflows with conditional logic. Ideal for VCC scenarios where feedback analysis leads to different form adaptation strategies based on detected patterns, and where the AI decision process needs explicit modeling and debugging.

Framework Comparison Matrix

Criteria	NanoBot	Opencode SDK	CrewAI	LangGraph
Learning Curve	Low	Medium	Medium	High
Setup Speed	Very Fast	Fast	Medium	Medium
Multi-Agent	Manual	Limited	Excellent	Good

Criteria	NanoBot	Opencode SDK	CrewAI	LangGraph
State Management	Basic	Good	Shared	Excellent
Production Ready	Moderate	High	High	High
Scalability	Medium	High	High	Very High

Table 2: Framework Comparison Matrix

Phase 1: Competitive Framework Testing

Objective

Each team member will select one framework (NanoBot, Opencode SDK, CrewAI, or LangGraph) to build a functional prototype of the VCC. This parallel development approach enables direct comparison of framework capabilities, development velocity, and AI reasoning quality under identical requirements.

Duration

Days 1-5 of the 2-week sprint. This phase allocates approximately 60% of the total project timeline to ensure thorough framework exploration and functional prototype completion.

Module 1: Check-In Module

The Check-In Module serves as the initial touchpoint for clients entering the Virtual Customer Care Office. The agent presents a web page or form for the client to register their arrival, collecting essential information that drives subsequent interactions.

Functional Requirements

- Client Registration Form: Capture name, email, phone number, purpose of visit, and company affiliation through a clean, accessible interface
- Purpose Selection: Provide predefined categories (General Inquiry, Account Services, Technical Support, Billing Question, Product Demo, Partnership Discussion) with option for custom entry
- Timestamp Recording: Automatically log check-in time for analytics and wait time calculations
- Confirmation Display: Show acknowledgment message with estimated wait time and queue position

Module 2: Check-Out Module

The Check-Out Module captures client feedback upon completion of their visit. This module is critical for data collection that fuels the AI analysis engine and drives form adaptation decisions.

Functional Requirements

- Rating System: Implement 1-5 star rating with optional hover descriptions for each level (Poor, Fair, Good, Very Good, Excellent)
- Comment Field: Provide free-text area for detailed feedback with character counter (minimum 10 characters, maximum 500)

- Dynamic Questions: Display AI-generated questions based on client's check-in purpose and previous feedback patterns
- Sentiment Capture: Automatically analyze comment sentiment upon submission for real-time categorization

Phase 2: Self-Evolving Feedback System

Objective

The core of this project is the AI's ability to autonomously refine its own data collection process. This phase implements the intelligence layer that transforms static forms into dynamic, contextually-aware instruments that evolve based on accumulated feedback.

Duration

Days 6-9 of the 2-week sprint. This phase runs concurrently with Phase 1 data collection once initial feedback is available, enabling continuous improvement cycles.

24-Hour Review Cycle

Every 24 hours, the AI must analyze all submitted feedback from the previous day. This scheduled analysis identifies patterns, detects anomalies, and generates actionable insights for form optimization. The cycle operates autonomously but logs all decisions for human review through the dashboard.

Dynamic Form Adaptation

Add Questions

If the AI identifies a recurring issue or a gap in information, it must automatically add a relevant question to the form. For example, if multiple clients mention long wait times, the AI adds a question asking clients to estimate their wait duration. This ensures the system captures data points that matter most to improving customer experience.

Remove Questions

If a question is no longer yielding useful data or is redundant, the AI must remove it. This prevents form fatigue and ensures clients only answer questions that provide actionable insights. Questions with consistently high skip rates or non-discriminating responses are candidates for removal.

Contextual Intelligence

The form should feel "alive" - it tells the next client what it has learned or asks deeper questions based on trends from past clients. This creates a conversational, intelligent experience that demonstrates the system's learning capabilities while gathering increasingly targeted feedback.

Phase 3: Internal Analytics Dashboard

Objective

While the client sees the form, the team needs a "behind-the-scenes" look at the AI's logic. This phase delivers a comprehensive dashboard that provides transparency into AI decision-making, displays actionable insights, and enables data-driven operational improvements.

Duration

Days 8-12 of the 2-week sprint. Development overlaps with Phase 2 to ensure dashboard readiness when AI analysis results become available.

Dashboard Components

Logic Visualization

A dedicated panel displays why the AI changed specific questions, showing the reasoning chain from feedback pattern detection to question modification. Each change includes supporting evidence such as the number of similar comments, sentiment scores, and confidence level of the AI's decision.

Sentiment & Trend Analysis

A summary section presents what the AI analyzed from customer feedback, including sentiment distribution charts (positive/neutral/negative), keyword frequency clouds, and trend lines showing satisfaction changes over time. This enables rapid identification of emerging issues or improvement opportunities.

Operational Insights

Clear indicators show where the business needs to improve based on AI findings. Example: "Clients consistently mention long wait times at 2 PM" triggers a staffing recommendation. These insights translate raw data into actionable business intelligence.

Phase 4: Evaluation & Selection

Objective

At the end of the two-week sprint, the team will compare the prototypes built using different frameworks to determine which approach best meets project requirements and organizational capabilities for future production deployment.

Duration

Days 13-14 of the 2-week sprint. This final phase synthesizes findings from all previous phases into actionable recommendations.

Evaluation Criteria

Performance Metrics

- Framework Handling: Which framework handled the implementation requirements best? Consider API response times, memory usage, and reliability under simulated load
- AI Reasoning Quality: Which AI logic was most "human-like" and accurate in updating the forms? Evaluate sentiment analysis accuracy and question relevance
- User Experience: Which prototype provided the most intuitive, responsive, and accessible interface for both clients and administrators?

Scalability Assessment

- Template Potential: Which version is easiest to turn into a template for different business products? Consider abstraction level and customization options
- Maintenance Overhead: Which framework requires the least ongoing maintenance and provides the clearest upgrade path?
- Team Expertise: Which framework aligns best with existing team skills and reduces the learning curve for future development?

Technical Architecture

Technology Stack

Layer	Technology	Purpose
Frontend	Next.js 15, React 19, Tailwind CSS	Responsive UI with real-time updates
Backend	Next.js API Routes, Prisma ORM	RESTful API with type-safe database access
Database	SQLite (dev), PostgreSQL (prod)	Persistent storage with migrations
AI Engine	z-ai-web-dev-sdk, OpenAI API	Sentiment analysis and pattern recognition
State Management	Zustand, React Query	Global state and server state caching
Visualization	Recharts	Interactive charts and data visualization

Table 3: Technology Stack Overview

Project Timeline

Day	Phase	Activities	Deliverable
1-2	Phase 1	Setup, Check-In Module	Working check-in form
3-5	Phase 1	Check-Out Module, Basic Analytics	Complete feedback flow
6-7	Phase 2	AI Integration, Sentiment Analysis	Working AI analysis
8-9	Phase 2	Dynamic Form Adaptation	Auto-updating forms
10-12	Phase 3	Analytics Dashboard Development	Complete dashboard
13-14	Phase 4	Testing, Evaluation, Documentation	Final report & recommendation

Table 4: Project Timeline Overview

Risk Assessment

Risk	Impact	Probability	Mitigation
Framework limitations	High	Medium	Parallel framework evaluation
AI accuracy issues	Medium	Medium	Fallback logic, human review
Timeline overrun	High	Low	MVP prioritization, daily standups
Data privacy concerns	High	Low	Anonymization, local storage

Table 5: Risk Assessment Matrix

Success Metrics

Quantitative Metrics

1. Sentiment Analysis Accuracy: Target >85% accuracy in classifying customer feedback sentiment (positive, neutral, negative) compared to human evaluation
2. Form Adaptation Relevance: Target >80% of AI-suggested questions rated as relevant by business stakeholders during review sessions
3. User Completion Rate: Target >90% of clients who check-in also complete the checkout feedback process
4. System Uptime: Target 99.5% availability during the testing phase with <2 second average response time

Qualitative Metrics

1. Development Velocity: Time to complete each phase measured against baseline estimates, with daily progress tracking
2. Code Quality: Maintainability score based on code review feedback, documentation completeness, and test coverage
3. User Experience: Stakeholder satisfaction ratings from usability testing sessions and feedback surveys
4. Learning Curve: Time required for new team members to understand and modify each framework's implementation

Conclusion

This Virtual Customer Care Office project represents an innovative approach to customer feedback management, leveraging AI to create a truly adaptive and intelligent system. The two-week sprint provides sufficient time to validate core concepts while maintaining focus on deliverable outcomes. Success in this proof-of-concept phase will inform decisions about production deployment and framework selection for future iterations.

The competitive framework testing approach ensures that the final recommendation is based on practical implementation experience rather than theoretical comparisons. By having team members

work with different frameworks simultaneously, we gain diverse perspectives on each technology's strengths and limitations in the context of our specific requirements.

Upon successful completion, the VCC system will demonstrate the feasibility of AI-driven customer service automation and provide a solid foundation for scaling to production environments across multiple business units.